Creating Northstar App for Suunto Watches

Development Flow with SuuntoPlus Sports Apps

To start building SuuntoPlus Sports Apps, developers need to join the Suunto Partner Program. Once accepted, they gain access to developer tools and documentation. While the API Zone submission process is still evolving, developers can already build and test their apps locally using Suunto's tooling. SuuntoPlus apps are structured projects with defined inputs, outputs, and logic, packaged for deployment on Suunto devices.

- **Join** the Suunto Partner Program to get an account for Suunto Apizone
- **Install** the SuuntoPlus Editor
- **Develop** the SuuntoPlus Sports apps locally with your own watch & computer
- **Submit** Once development is ready, submit the SuuntoPlus sports app via ApiZone own Profile page
- **Publish:** Suunto will publish your sports app after review of the Suuntoplus Sports app and related content
- **Update:** Once the SuuntoPlus sports app is published, you will get direct feedback via Email used in your Apizone account.

2. How to Start with Suunto Cloud API

After your SuuntoPlus watch app is created, the next step is integrating the backend using Suunto Cloud API. This allows you to retrieve user workouts, sync routes, store hiking data, and build watch-to-mobile features. Click here to start

Hep Request (SOS) Implementation Flow (Watch to Mobile)

The custom SuuntoPlus App acts as a JavaScript-based client that runs on the watch *during an active sport mode* (e.g., Hiking).

1. **Watch App Logic (JavaScript):** You will code the SuuntoPlus App to monitor the watch state for the specific SOS activation event (e.g., a long press of the lower button, or a designated screen tap within your app's display).
2. **GPS Capture:** When the SOS event is detected, the app uses the internal API bindings provided by the SuuntoPlus SDK to capture the watch's current **GPS coordinates** (latitude, longitude, altitude) immediately.
3. **Real-Time Data Injection:** This is the most critical step. Instead of waiting for a full cloud sync, your SuuntoPlus App must use the internal mechanism (often related to the **Companion API** access granted to partners for live-tracking) to inject a custom, high-priority data packet directly to the paired mobile phone app.
    a. **Data Payload:** This packet must contain the unique identifier for the user, the GPS location, and a distinct flag indicating it is an emergency: {"event": "SOS_INITIATED", "timestamp": [Time], "lat": [Lat], "lon": [Lon]}.

II. Mobile-Side Development: The Bridge and Listener

You must build a custom mobile application (iOS/Android) that is constantly running in the background and is authorized by Suunto to receive the real-time stream.

1. **Custom Mobile App SDK Integration:** Your mobile app development (using React native) must integrate the low-level SDK provided by Suunto to partners. This SDK allows your app to establish a persistent Bluetooth Low Energy (BLE) channel with the watch and listen to the real-time data stream coming from your custom SuuntoPlus App.
2. **Immediate Backend Trigger:** Your mobile app's listener logic is paramount:

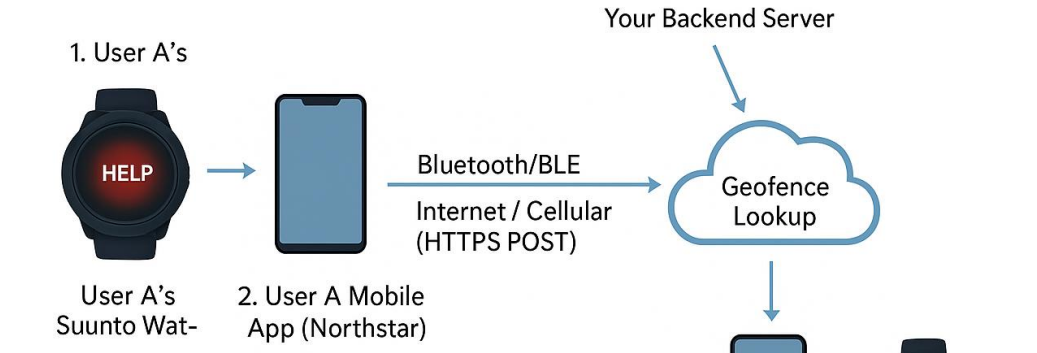
a. It waits for the data packet from the watch.
b. When the SOS_INITIATED flag is received, the app immediately executes a standard internet request (HTTP POST) to your own secure backend server, including the captured GPS coordinates. This bypasses the slower workout cloud sync entirely.

III. Backend Integration and Alerting

Your server handles the logic of finding and notifying nearby users.

1. **API Used: SUUNTO AUTHORISATION API (OAuth 2):** Use this to secure the user's connection and data access. Users must authorize your app to access their Suunto data (even if that access is limited to real-time SOS flags).
2. **API Used: Your Own Custom REST API:** Your backend must have an endpoint (/api/help) designed to receive the high-priority SOS request from the user's mobile app.
3. **Alerting Flow:**
    a. Your backend receives the SOS request (User ID, Lat/Lon).
    b. It performs a Geo-location lookup on your user database to identify all users within a designated radius (e.g., 5km).
    c. It sends real-time push notifications using standard mobile infrastructure (FCM/APNS) to the nearby "responder" phones.
4. **Responder Watch Alert:** When a nearby user's phone receives your FCM/APNS notification, the official Suunto App's built-in functionality takes over, mirroring your notification text (e.g., "EMERGENCY: User A needs help at 40.7, -74.0") directly to the responder's Suunto watch display.

Northstar App: Help Request Flow



⚠ CRITICAL LIMITATION:

- Phone has no internet connection
- Watch is not connected to phone